

PostgreSQL 19 **SQL/PGQ**: When Similarity Alone Is Not Enough

Rajesh Madiwale

Sr Solution Architect · Yugabyte

AGENDA

What we will understand together

1. **SQL/PGQ** — the new graph feature in PostgreSQL 19
2. **The challenge before it** — how we answered relationship questions until now
3. **A real problem** that SQL or vectors alone cannot solve
4. **The difference** between SQL, pgvector and SQL/PGQ
5. **A live demo** — one question, three very different answers

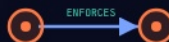
POSTGRESQL 19 · NEW FEATURE

What is SQL/PGQ?

SQL/PGQ means **Property Graph Queries inside SQL**. We describe tables we already have as **vertices** and **edges**, then search for relationship **patterns** using `GRAPH_TABLE` and `MATCH`.

VERTEX**A business object**

A person, a policy, a control, a database.

[CLICK TO DEFINE](#)**EDGE****A relationship**

Applies to · enforces · protects.

[CLICK TO DEFINE](#)**PATTERN****The question shape**

Who is connected to what, and how?

[CLICK TO DEFINE](#)

WHY IT MATTERS

Advantages of SQL/PGQ

Relationship questions stay readable

The query shows the path itself — policy, control, system — instead of hiding it inside a long chain of joins.

Your tables remain the source of truth

A property graph is only a definition placed over tables you already have. There is no second copy to keep in step.

Results are ordinary SQL rows

GRAPH_TABLE returns a normal result set, so you can filter, join, aggregate and report on it as usual.

It is a standard, not a new language

SQL/PGQ comes from the SQL:2023 standard, so the skill your team learns is transferable, not vendor-specific.

In one line: we can finally ask relationship questions while keeping PostgreSQL as the single source of truth.

BEFORE POSTGRESQL 19

How did we answer these questions until now?

STAY INSIDE POSTGRESQL

- Chain several self-joins together for one fixed path
- Move to recursive CTEs as soon as the depth can change
- Every new question means writing long SQL that is hard to review

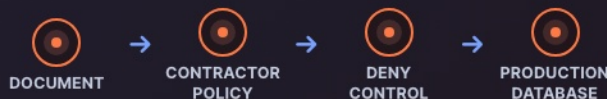
GO OUTSIDE POSTGRESQL

- Copy the same data into a separate graph database
- Build and monitor pipelines to keep both systems in step
- Run, secure, back up and staff a second platform

The real challenge: the business asks a simple relationship question, and we answer it either with complicated SQL or by adding another database.

A REAL PROBLEM · AI ACCESS ASSISTANT

"Can contractors access production databases?"



The wording problem

The user asks about "contractors", but the policy may call them "third-party staff".

The relationship problem

No single document states the decision. It only exists across several linked records.

The trust problem

For a security question, a document that merely looks relevant is not an acceptable answer.

What we actually need: not just the word "No", but the evidence path that proves it.

UNDERSTANDING THE DIFFERENCE

Normal SQL vs pgvector vs SQL/PGQ

NORMAL SQL

**Finds exact facts**

Filters, keys, joins and transactions.
This is where the trusted data lives.

On its own: it misses different wording, and long paths become long joins.

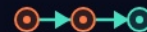
PGVECTOR

**Finds similar meaning**

Compares embeddings, so
"contractor" can still match "third-party staff".

On its own: it returns a relevant document, never the decision behind it.

SQL/PGQ

**Finds connected facts**

Matches a relationship pattern and
returns every path that fits it.

What it adds: the evidence path that explains the answer.

Used together: SQL keeps the trusted facts, pgvector finds the right starting point, and SQL/PGQ follows the evidence.

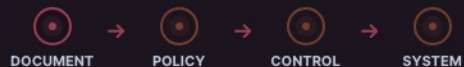
DEMO 1 · NORMAL SQL

Searching by keywords

"Can contractors access production databases?"

```
SELECT id, title, body  
FROM documents  
WHERE body ILIKE '%contractor%production%';
```

WHAT WE SEE



No rows returned. The policy exists — our keywords simply did not match its wording.

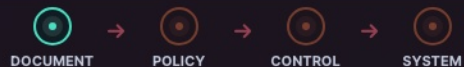
DEMO 2 · PGVECTOR

Searching by meaning

"Can contractors access production databases?"

```
SELECT id, title  
FROM documents  
ORDER BY embedding ⇔ '[0.88, 0.12, 0.10]'  
LIMIT 1;
```

WHAT WE SEE



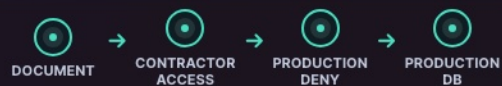
Finds "Access control overview", but it only says "see the linked policies". The rest of the path is still dark.

DEMO 3 · PGVECTOR + SQL/PGQ

Following the relationships

```
SELECT *  
FROM GRAPH_TABLE (  
  access_kg  
  MATCH (doc IS document WHERE doc.id = 1)  
    -[IS mentions]→(pol IS entity)  
    -[IS link]→(ctl IS entity)  
    -[IS link]→(sys IS entity)  
  COLUMNS (pol.name, ctl.name, sys.name)  
);
```

THE EVIDENCE PATH



Answer: No — contractors cannot access production databases, and we can show exactly why.

WHAT WE GAINED

Benefits we just demonstrated

The answer can be explained

We return the reasoning path, not only a document that looked similar.

The query reads like the question

Policy, control and system are visible, so a reviewer can confirm the logic.

One platform, three kinds of search

Exact facts, meaning and relationships all answered inside PostgreSQL.

Less data movement

We used relationships that were already sitting in our tables.



Thank you

Rajesh Madiwale

Sr Solution Architect · Yugabyte

QUESTIONS WELCOME · PGDAY 2026

To remember

- SQL finds exact facts
- pgvector finds similar meaning
- SQL/PGQ finds and explains relationships